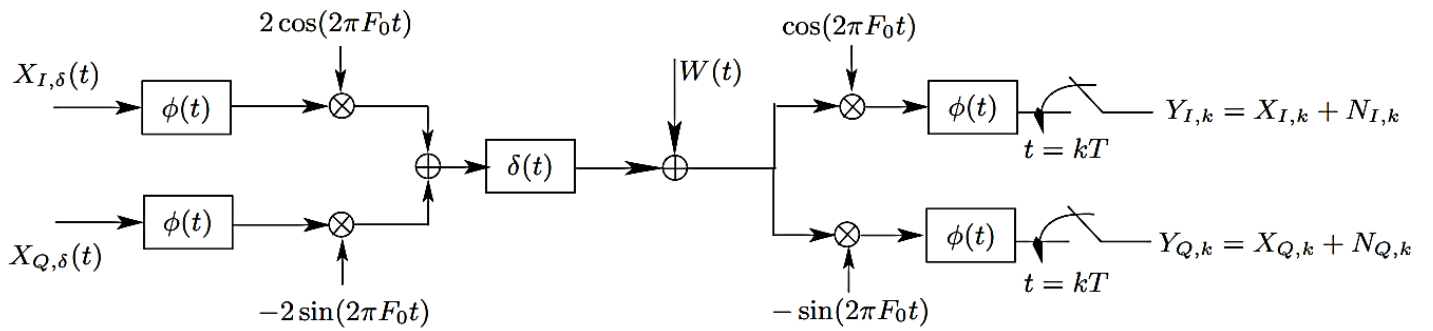

Technical University of Crete
School of Electrical and Computer Engineering
Telecommunication Systems I
Instructor: Athanasios P. Liavas

Exercise 3
PATRIKAKI ARISTEA 2023030078

Hours required: 5,5 hours



In this exercise, we will simulate the telecommunication system shown in the Figure, assuming that 16-QAM modulation is used, and we will study its performance. More specifically, we will implement the 16-QAM system as two independent 4-PAM systems, with constellation $\{\pm A, \pm 3A\}$.

1. For a given N (for example, $N = 200$), generate a binary sequence with $4N$ equiprobable bits.
2. (15) Write a function

`function X = bits_to_4_PAM(bit_seq, A)`

which, using Gray coding, maps the input binary sequence `bit_seq` to a sequence of 4-PAM symbols.

3. Map the first $2N$ bits of the sequence from Step 1 to the 4-PAM symbols $X_{I,n}$, for $n = 1, \dots, N$, and the next $2N$ bits to the 4-PAM symbols $X_{Q,n}$, for $n = 1, \dots, N$.

MATLAB code:

```
clear; close all; clc;

%% Parameters
N = 200;
Aamp = 1;

%% 1. Generate 4N random bits
bits = randi([0 1], 1, 4*N);

%% 3. Split bits and map to 4-PAM
bitsI = bits(1:2*N);
bitsQ = bits(2*N+1:end);

XI_sym = bits_to_4PAM(bitsI, Aamp);
XQ_sym = bits_to_4PAM(bitsQ, Aamp);
```

```
function X = bits_to_4PAM(bit_seq, A)
    bit_seq = bit_seq(:).';
    bit_pairs = reshape(bit_seq, 2, []).';
    X = zeros(1, size(bit_pairs,1));

    for k = 1:size(bit_pairs,1)
        b1 = bit_pairs(k,1);
        b2 = bit_pairs(k,2);

        if b1 == 0 && b2 == 0
            X(k) = -3*A;
        elseif b1 == 0 && b2 == 1
            X(k) = -A;
        elseif b1 == 1 && b2 == 1
            X(k) = A;
        elseif b1 == 1 && b2 == 0
            X(k) = 3*A;
        end
    end
end
```

4. (5) Pass the sequences $\{X_{I,n}\}$ and $\{X_{Q,n}\}$ through the SRRC pulse-shaping filters. Assume a symbol period $T = 10^{-3}$ sec, over = 10, $T_s = \frac{T}{\text{over}}$ and construct and plot the output waveforms $X_I(t)$ and $X_Q(t)$ (**set the correct time axis**), as well as their periodograms.

MATLAB code:

```
clear; close all; clc;

%% Parameters
N = 200;
Aamp = 1; % PAM amplitude A
T = 10^(-3);
over = 10;
Ts = T/over;
Fs = 1/Ts;
A = 4;
a = 0.5;

Nf = 4096;
F = (-Nf/2:Nf/2-1) * Fs/Nf;

%% SRRC pulse
[phi, t_phi] = srrc_pulse(T, over, A, a);

%% 1. Generate 4N random bits
bits = randi([0 1], 1, 4*N);

%% 3. Split bits and map to 4-PAM
bitsI = bits(1:2*N);
bitsQ = bits(2*N+1:end);

XI_sym = bits_to_4PAM(bitsI, Aamp);
XQ_sym = bits_to_4PAM(bitsQ, Aamp);

%% 4. Pulse shaping
XI_delta = zeros(1, over*length(XI_sym));
XQ_delta = zeros(1, over*length(XQ_sym));

XI_delta(1:over:end) = XI_sym;
XQ_delta(1:over:end) = XQ_sym;

XI_delta = XI_delta / Ts;
XQ_delta = XQ_delta / Ts;

XI_t = conv(XI_delta, phi) * Ts;
XQ_t = conv(XQ_delta, phi) * Ts;

t = t_phi(1) + (0:length(XI_t)-1)*Ts;

figure;
```

```

plot(t, XI_t); grid on;
xlabel('t (s)'); ylabel('X_I(t)');
title('In-phase pulse-shaped waveform');

figure;
plot(t, XQ_t); grid on;
xlabel('t (s)'); ylabel('X_Q(t)');
title('Quadrature pulse-shaped waveform');

%% Periodogram of X_I(t)

XI_f = Ts * fftshift(fft(XI_t, Nf));
Ttotal = length(XI_t) * Ts;
PXI = abs(XI_f).^2 / Ttotal;

figure;
semilogy(F, PXI);
grid on;
xlabel('f (Hz)'); ylabel('Periodogram');
title('Periodogram of X_I(t)');

%% Periodogram of X_Q(t)

XQ_f = Ts * fftshift(fft(XQ_t, Nf));
Ttotal = length(XQ_t) * Ts;
PXQ = abs(XQ_f).^2 / Ttotal;

figure;
semilogy(F, PXQ);
grid on;
xlabel('f (Hz)'); ylabel('Periodogram');
title('Periodogram of X_Q(t)');

```

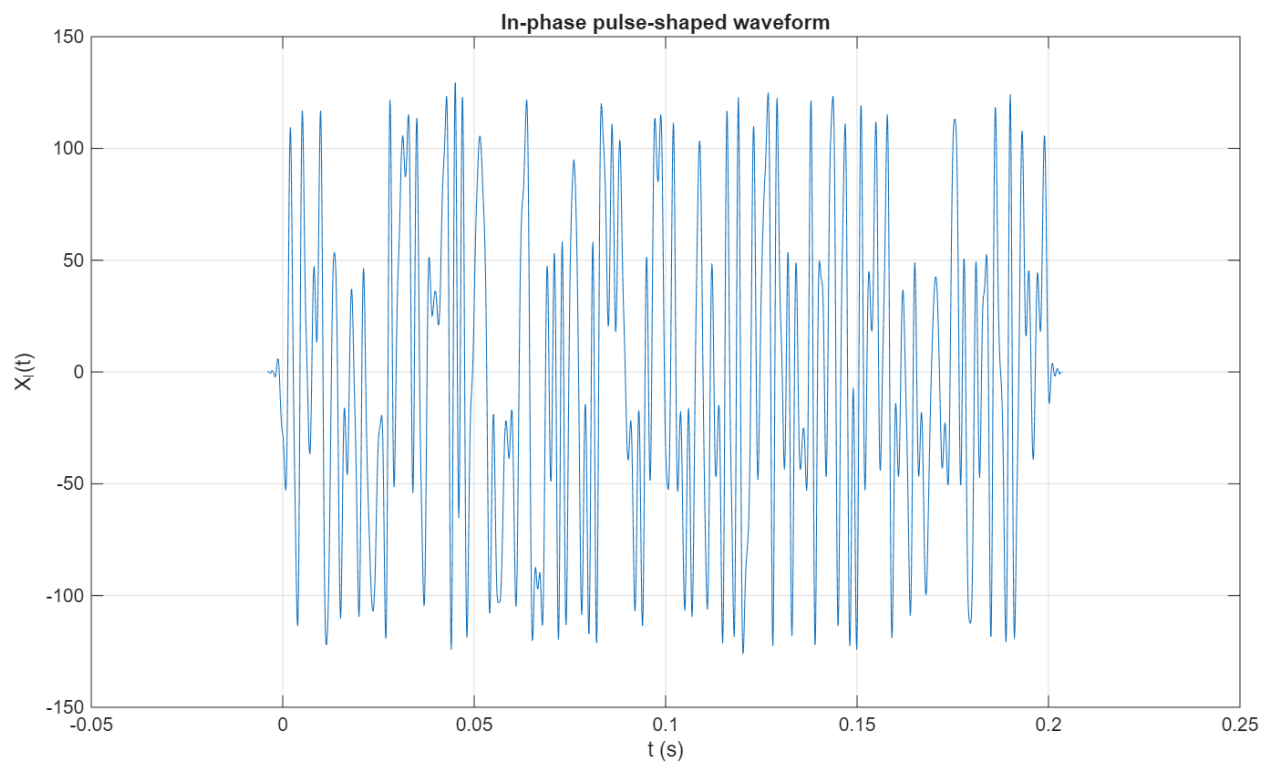


Figure 1. In-phase pulse-shaped waveform $X_I(t)$.

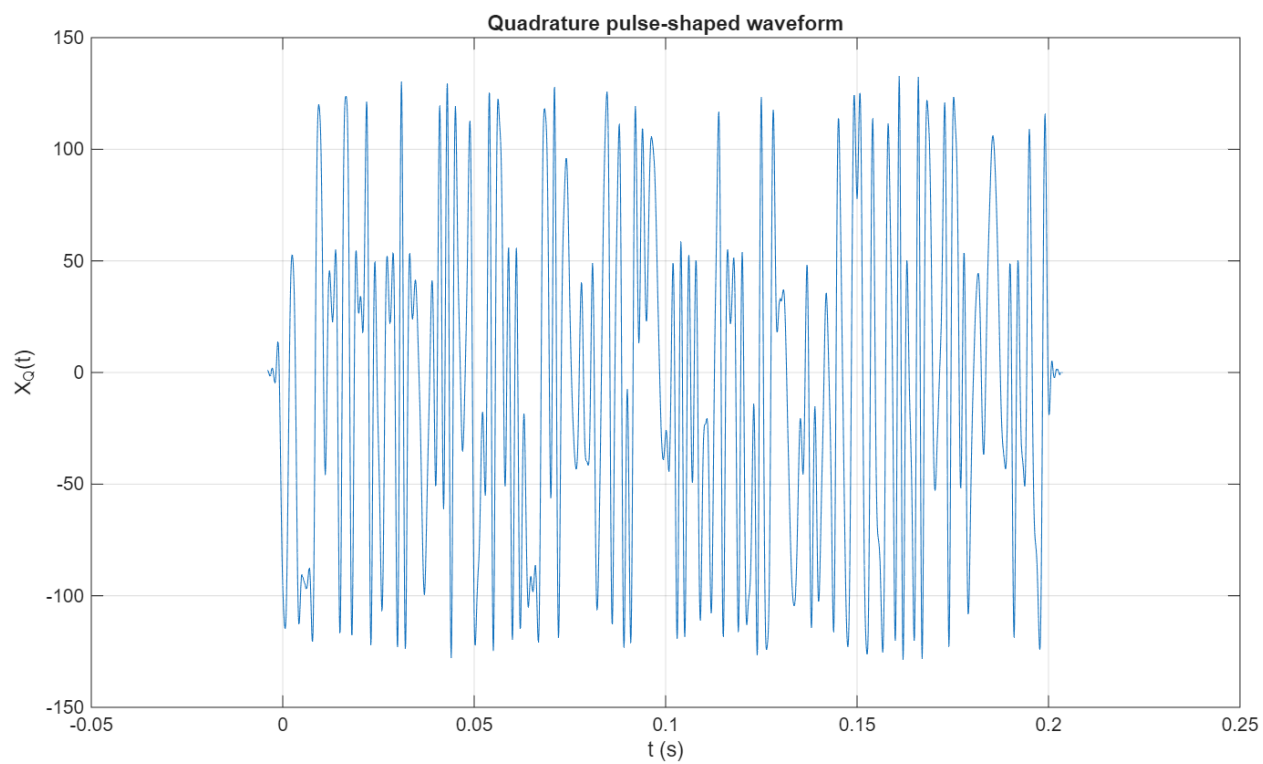


Figure 2. Quadrature pulse-shaped waveform $X_Q(t)$.

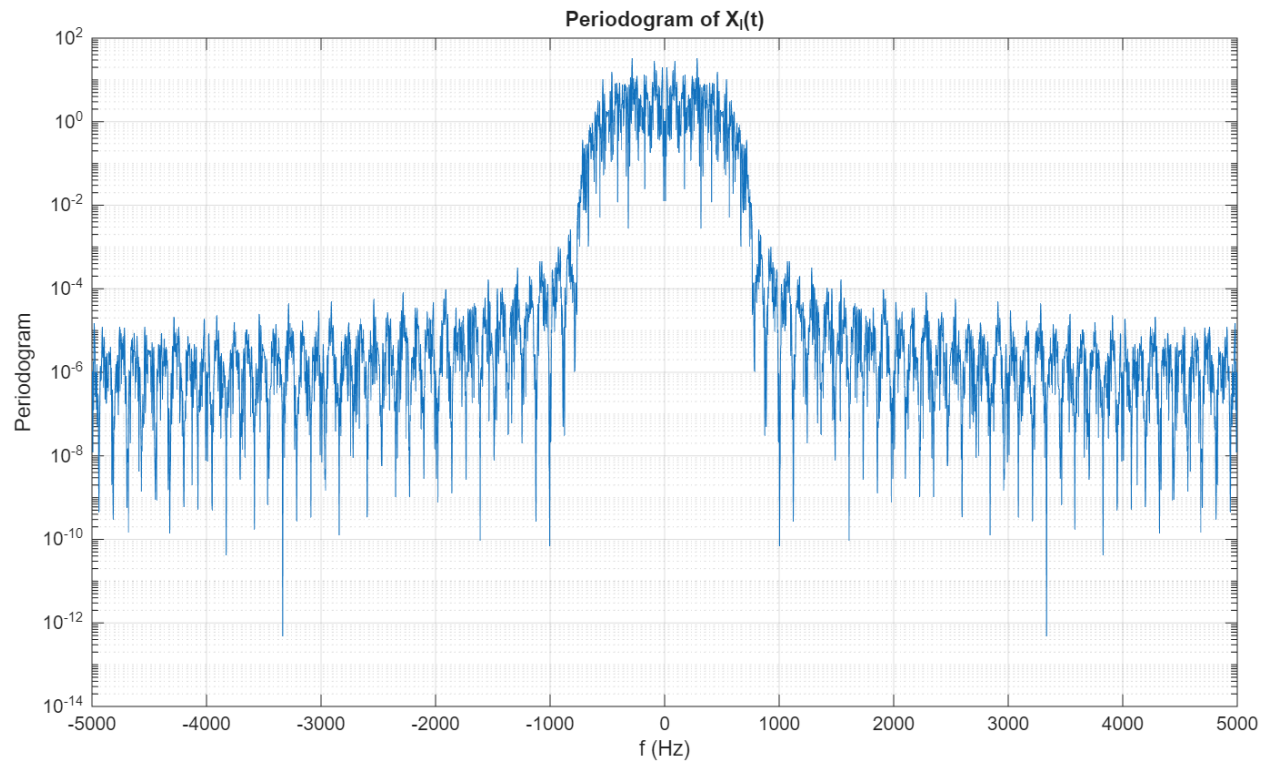


Figure 3. Periodogram of the in-phase baseband signal $X_I(t)$.

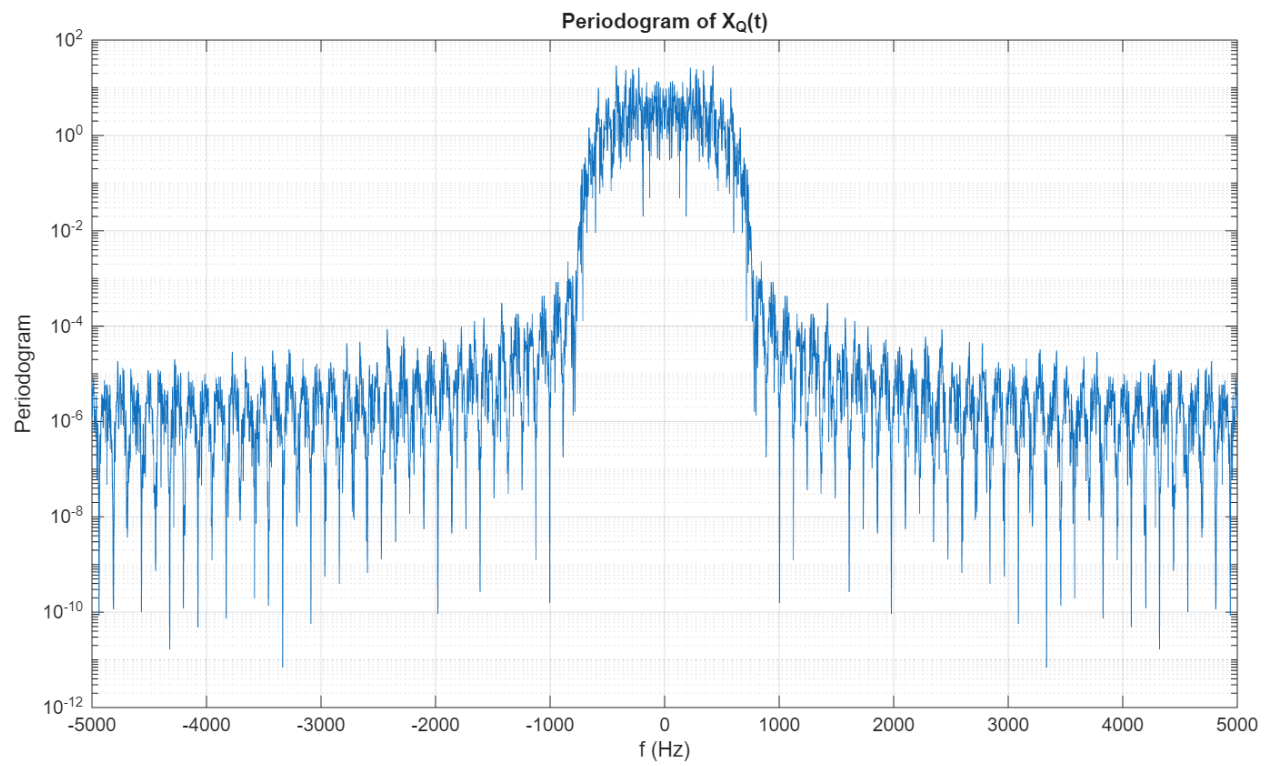


Figure 6. Periodogram of the quadrature baseband signal $X_Q(t)$.

5. (5) Multiply the waveforms $X_I(t)$ and $X_Q(t)$ by the corresponding carriers (for example, $F_0 = 2 \text{ KHz}$) and generate the waveforms

$$X_I^{\text{mod}}(t) = 2 X_I(t) \cos(2\pi F_0 t), \quad X_Q^{\text{mod}}(t) = -2 X_Q(t) \sin(2\pi F_0 t).$$

Plot the resulting waveforms as well as the corresponding periodograms. What do you observe?

MATLAB code:

```
%% 5. Modulation
F0 = 2000;           % 2 kHz

XI_mod = 2 * XI_t .* cos(2*pi*F0*t);
XQ_mod = -2 * XQ_t .* sin(2*pi*F0*t);

%% Plot X_I^mod(t)
figure;
plot(t, XI_mod); grid on;
xlabel('t (sec)'); ylabel('X_I^{mod}(t)');
title('Modulated in-phase signal');

%% Plot X_Q^mod(t)
figure;
plot(t, XQ_mod); grid on;
xlabel('t (sec)'); ylabel('X_Q^{mod}(t)');
title('Modulated quadrature signal');

%% Periodogram of X_I^mod(t)
XI_mod_f = Ts * fftshift(fft(XI_mod, Nf));
Ttotal = length(XI_mod) * Ts;
PXI_mod = abs(XI_mod_f).^2 / Ttotal;

figure;
semilogy(F, PXI_mod); grid on;
xlabel('f (Hz)'); ylabel('Periodogram');
title('Periodogram of X_I^{mod}(t)');

%% Periodogram of X_Q^mod(t)
XQ_mod_f = Ts * fftshift(fft(XQ_mod, Nf));
Ttotal = length(XQ_mod) * Ts;
PXQ_mod = abs(XQ_mod_f).^2 / Ttotal;

figure;
semilogy(F, PXQ_mod); grid on;
xlabel('f (Hz)'); ylabel('Periodogram');
title('Periodogram of X_Q^{mod}(t)');
```

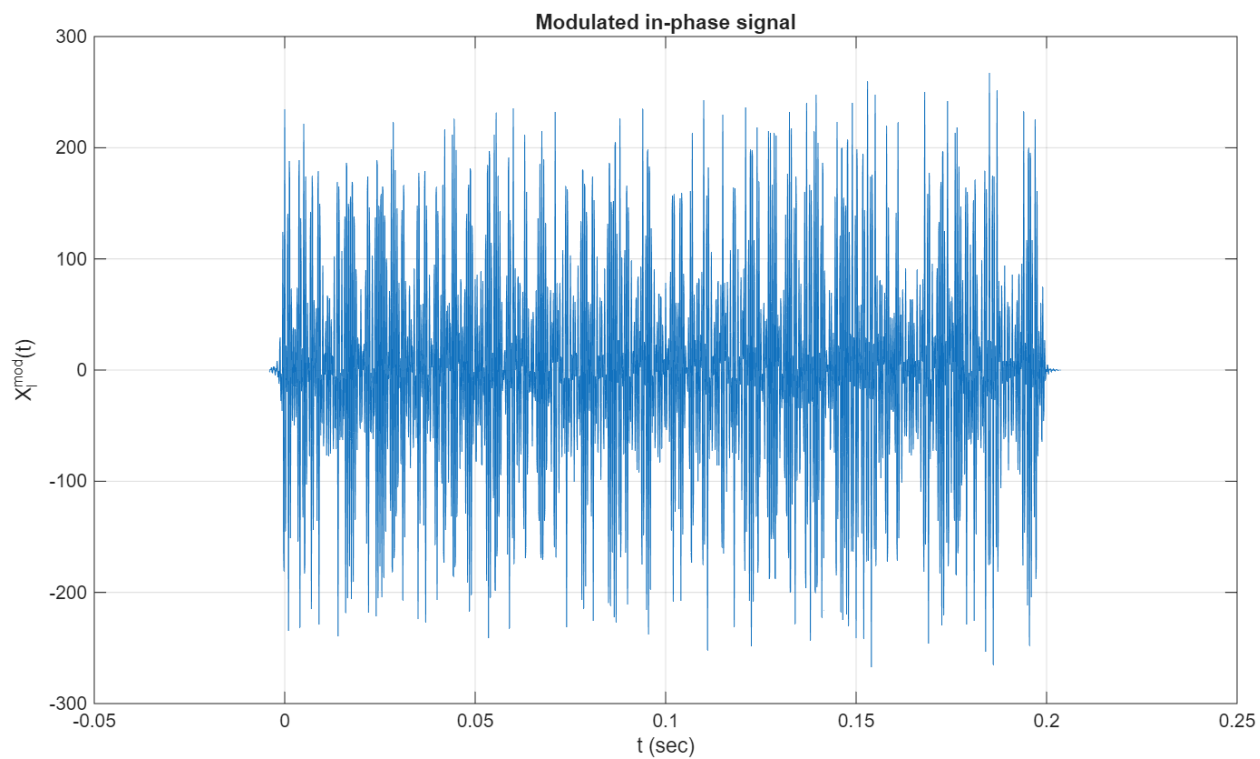


Figure 3. Modulated in-phase waveform $X_I^{\text{mod}}(t)$.

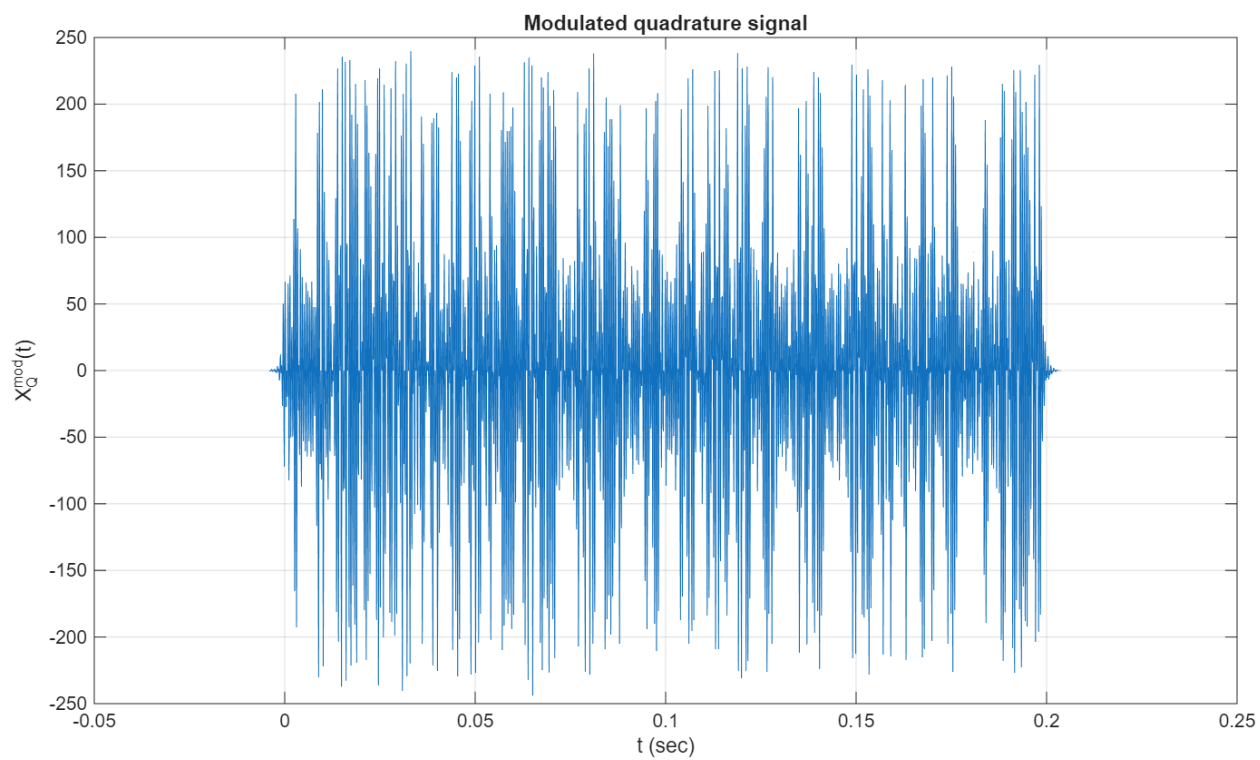


Figure 4. Modulated quadrature waveform $X_Q^{\text{mod}}(t)$.

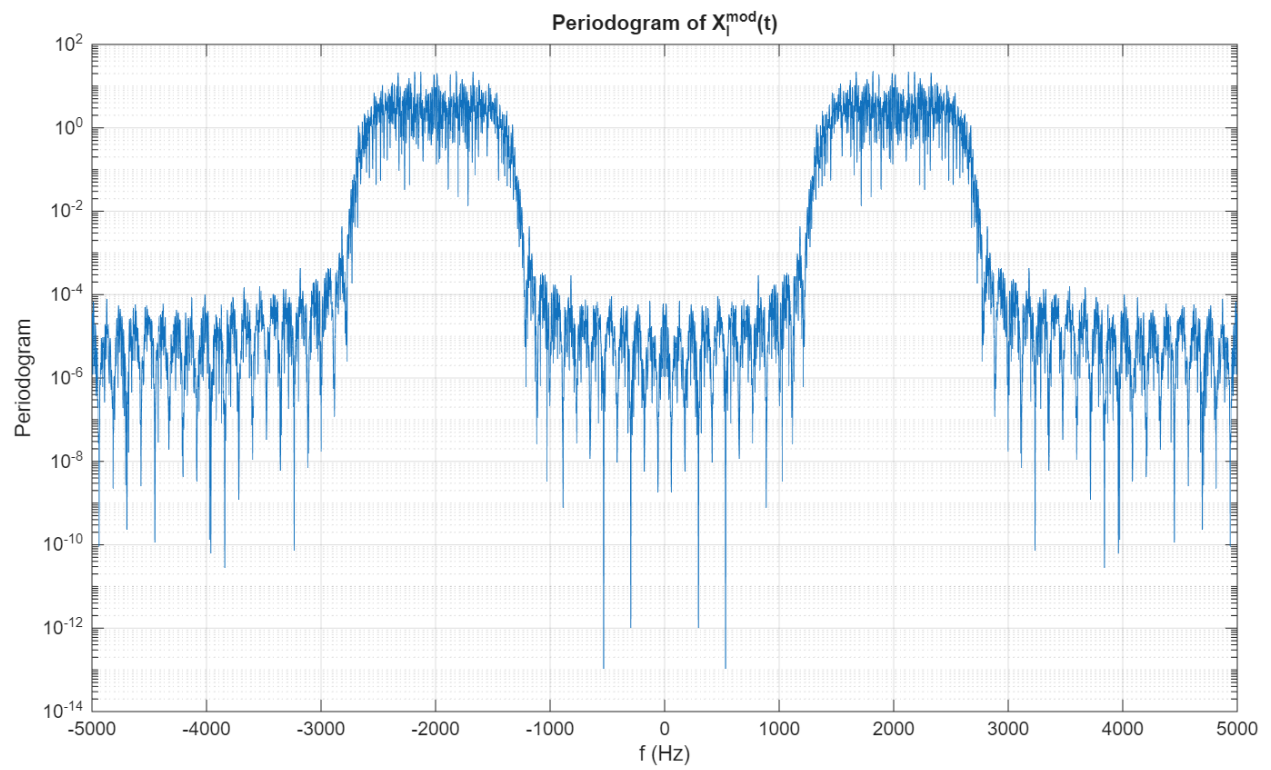


Figure 5. Periodogram of the modulated in-phase signal.

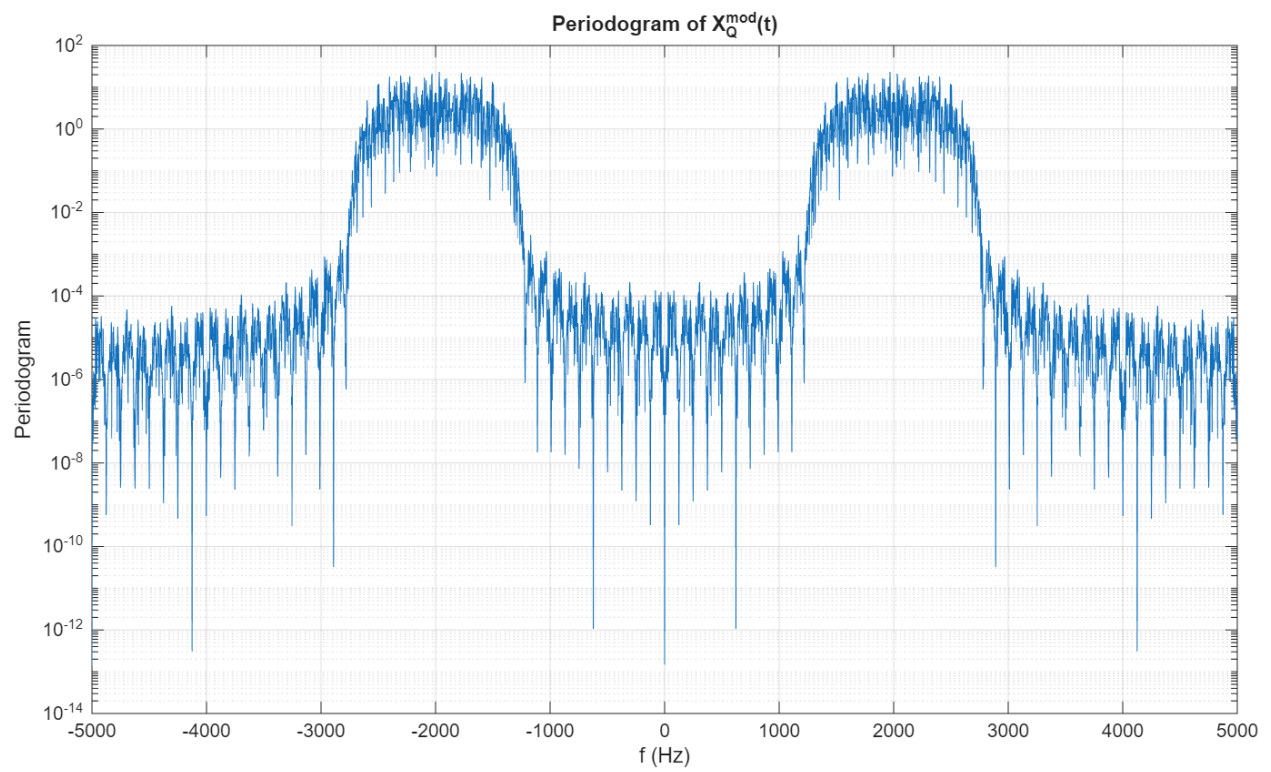


Figure 6. Periodogram of the modulated quadrature signal.

After multiplication by the carriers, the spectra of the signals are shifted from baseband to the frequencies around $\pm F_0$. Therefore, the information that was originally centered around 0 Hz is now transferred to the passband. The two modulated signals have similar spectral shapes, but one is multiplied by a cosine carrier and the other by a sine carrier, meaning that they correspond to orthogonal in-phase and quadrature components.

6. (5) Construct and plot the channel input,

$$X^{\text{mod}}(t) = X_I^{\text{mod}}(t) + X_Q^{\text{mod}}(t)$$

and its periodogram. What do you observe?

MATLAB code:

```
%% 6. Channel input signal

Xmod = XI_mod + XQ_mod;

%% Plot X^mod(t)
figure;
plot(t, Xmod);
grid on;
xlabel('t (sec)'); ylabel('X^{mod}(t)');
title('Channel input signal X^{mod}(t)');

%% Periodogram of X^mod(t)
Xmod_f = Ts * fftshift(fft(Xmod, Nf));
Ttotal = length(Xmod) * Ts;
PXmod = abs(Xmod_f).^2 / Ttotal;

figure;
semilogy(F, PXmod);
grid on;
xlabel('f (Hz)'); ylabel('Periodogram');
title('Periodogram of X^{mod}(t)');
```

In the time domain, the resulting waveform appears more complex because it contains both components simultaneously. In the frequency domain, its periodogram is still centered around the carrier frequencies $\pm F_0$, showing that the baseband information has been transferred to the passband. Since the two carriers are orthogonal, the I and Q components can be transmitted together without interfering with each other ideally.

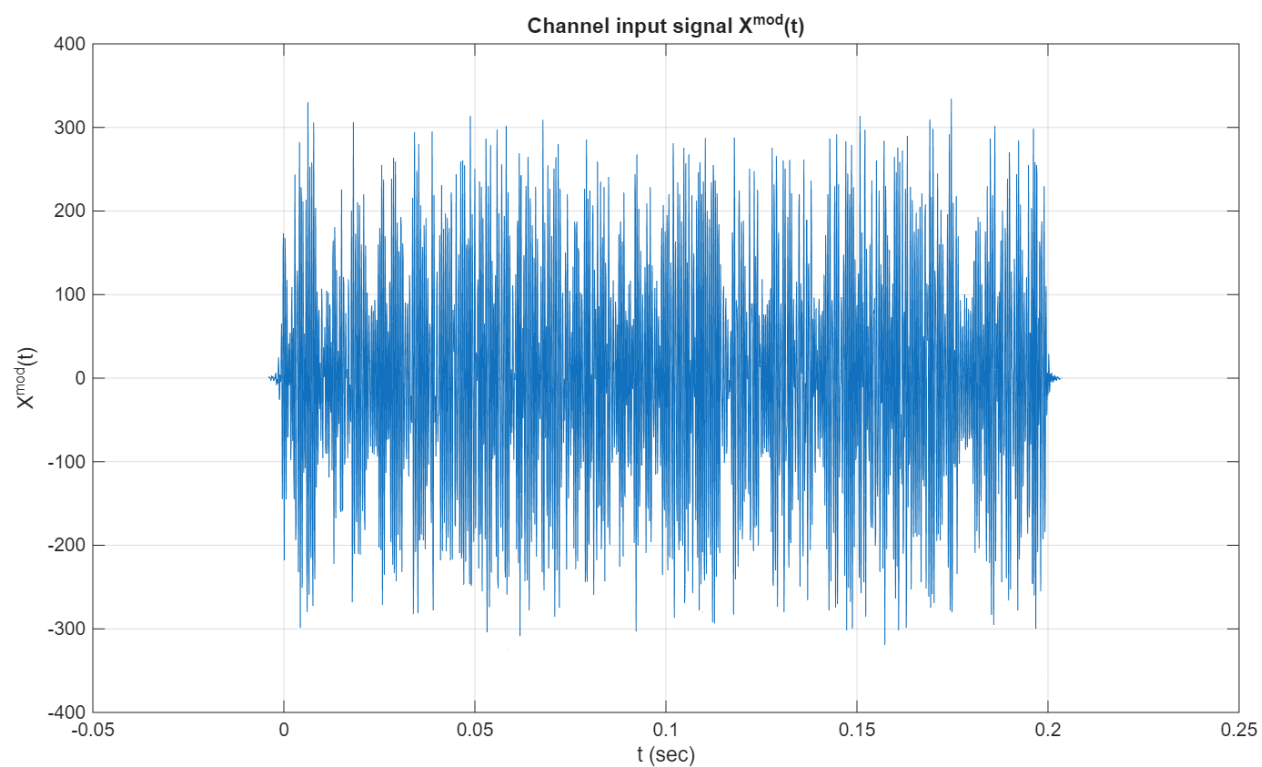


Figure 7. Channel input waveform $X^{\text{mod}}(t)$.

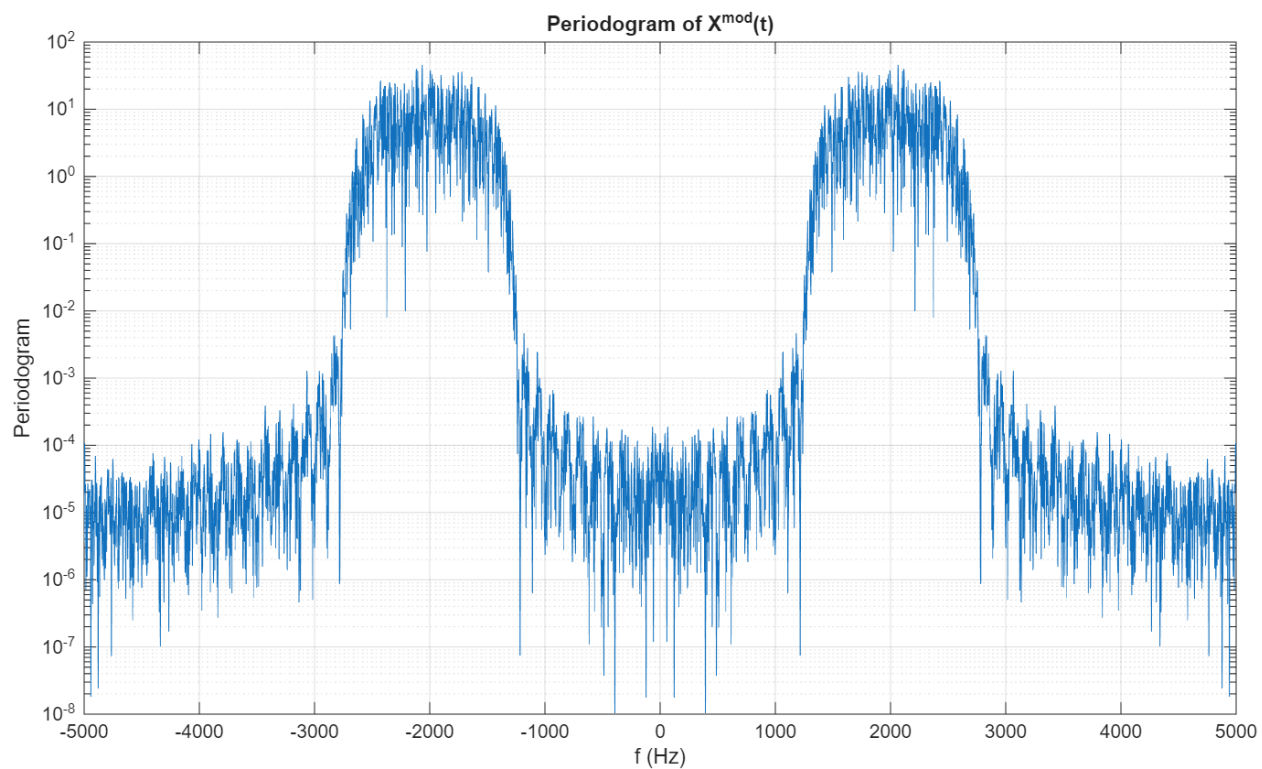


Figure 8. Periodogram of the channel input signal.

8. (5) At the channel output, add white Gaussian noise $W(t)$ with variance equal to

$$\sigma_W^2 = \frac{10A^2}{T_s \cdot 10^{\frac{SNR_{dB}}{10}}}.$$

MATLAB code:

```
% 8. AWGN noise
SNRdB = 10;

sigmaW2 = 10*Aamp^2 / (Ts * 10^(SNRdB/10));
W = sqrt(sigmaW2) * randn(size(Xmod));

Y = Xmod + W;

%% Verification of noise variance and SNR

sigmaN2 = Ts * sigmaW2 / 2;
SNR_check = 10*log10((10*Aamp^2)/(2*sigmaN2));

fprintf('Defined SNRdB = %.2f dB\n', SNRdB);
fprintf('sigmaW2 = %.4e\n', sigmaW2);
fprintf('sigmaN2 = %.4e\n', sigmaN2);
fprintf('SNR from formula = %.2f dB\n', SNR_check);
```

Results:

```
Defined SNRdB = 10.00 dB
sigmaW2 = 1.0000e+04
sigmaN2 = 5.0000e-01
SNR from formula = 10.00 dB
```

The computed value of $10\log_{10}(10A^2/(2\sigma_N^2))$ was equal to the predefined SNR_{dB} , confirming that the AWGN variance was correctly selected.

9. (5) At the receiver, split the noisy waveform into two branches and multiply them by the carriers $\cos(2\pi F_0 t)$ and $-\sin(2\pi F_0 t)$, respectively. Plot the waveforms that result and their periodograms. What do you observe?

MATLAB code:

```
%% 9. Receiver demodulation
YI_demod = Y .* cos(2*pi*F0*t);
YQ_demod = Y .* (-sin(2*pi*F0*t));

%% Plot I branch
figure;
plot(t, YI_demod);
grid on;
xlabel('t (sec)'); ylabel('Y_I(t)');
title('I branch after multiplication with cos(2\pi F_0 t)');

%% Plot Q branch
figure;
plot(t, YQ_demod);
grid on;
xlabel('t (sec)'); ylabel('Y_Q(t)');
title('Q branch after multiplication with -sin(2\pi F_0 t)');

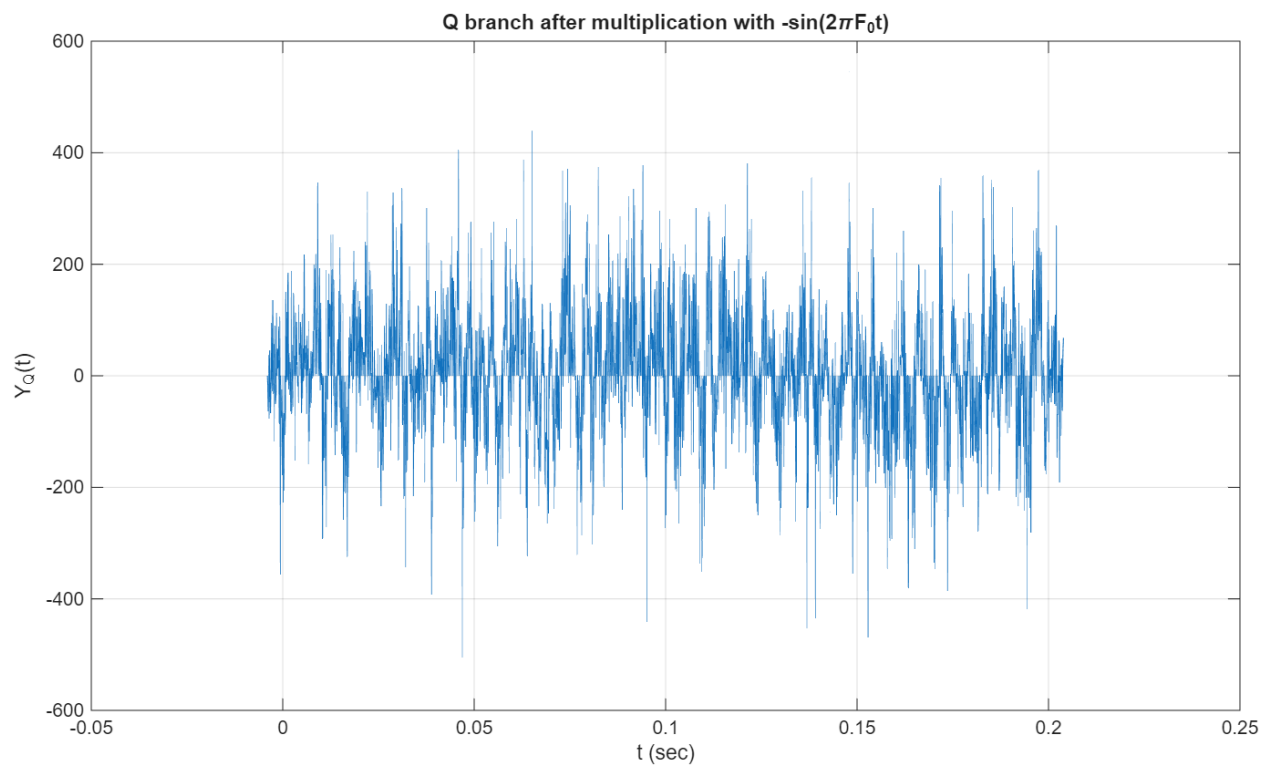
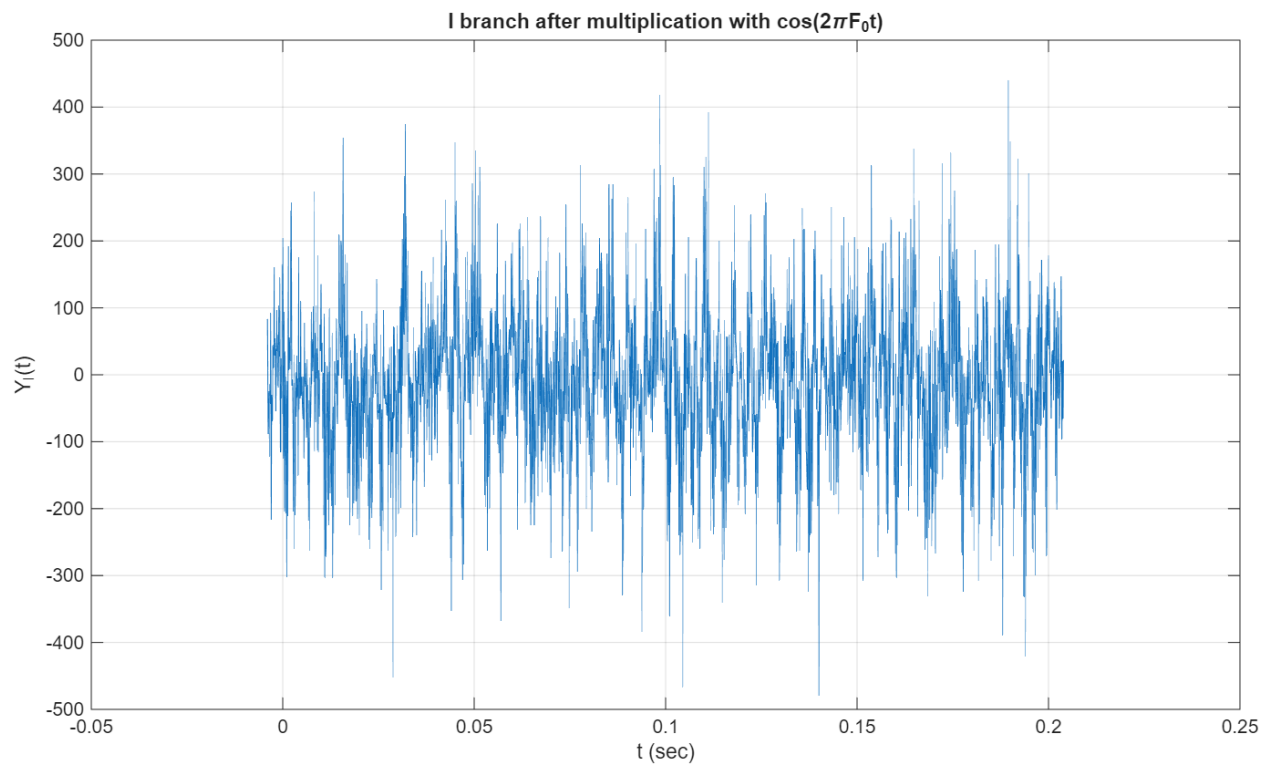
%% Periodogram of I branch
YI_demod_f = Ts * fftshift(fft(YI_demod, Nf));
Ttotal = length(YI_demod) * Ts;
PYI_demod = abs(YI_demod_f).^2 / Ttotal;

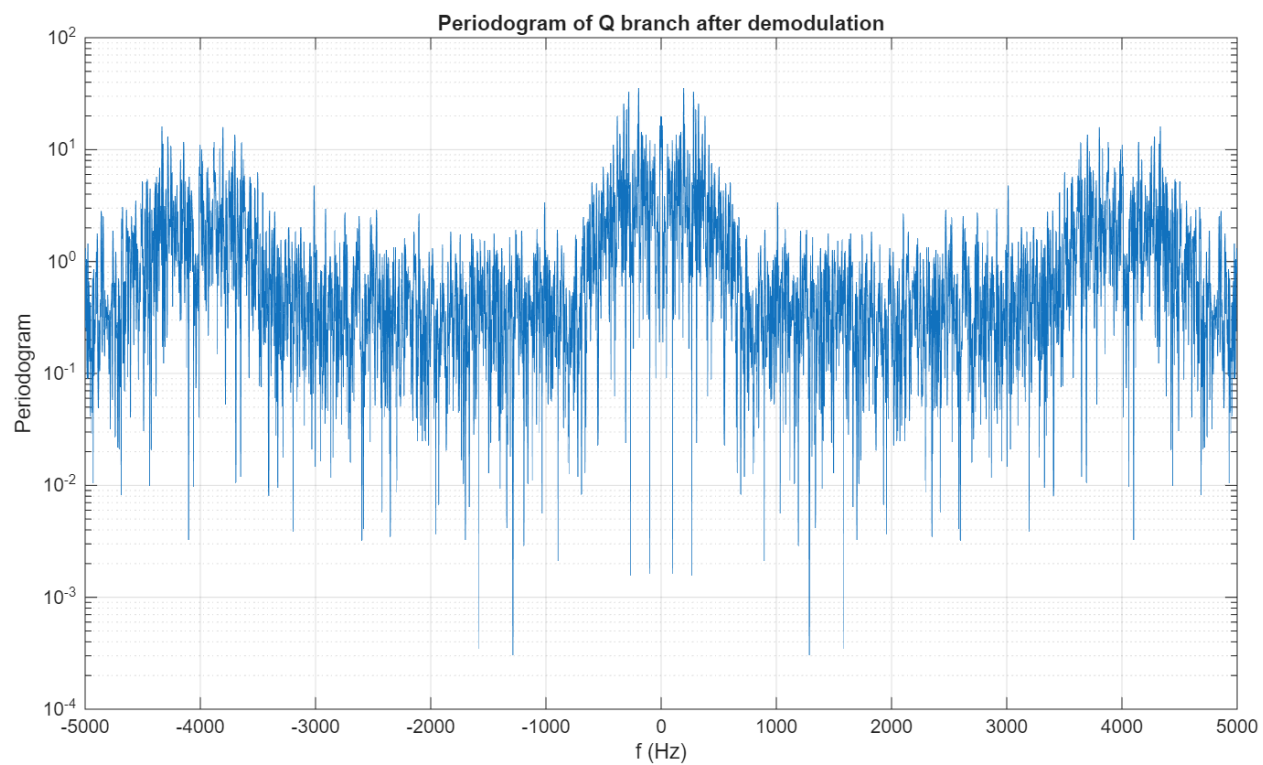
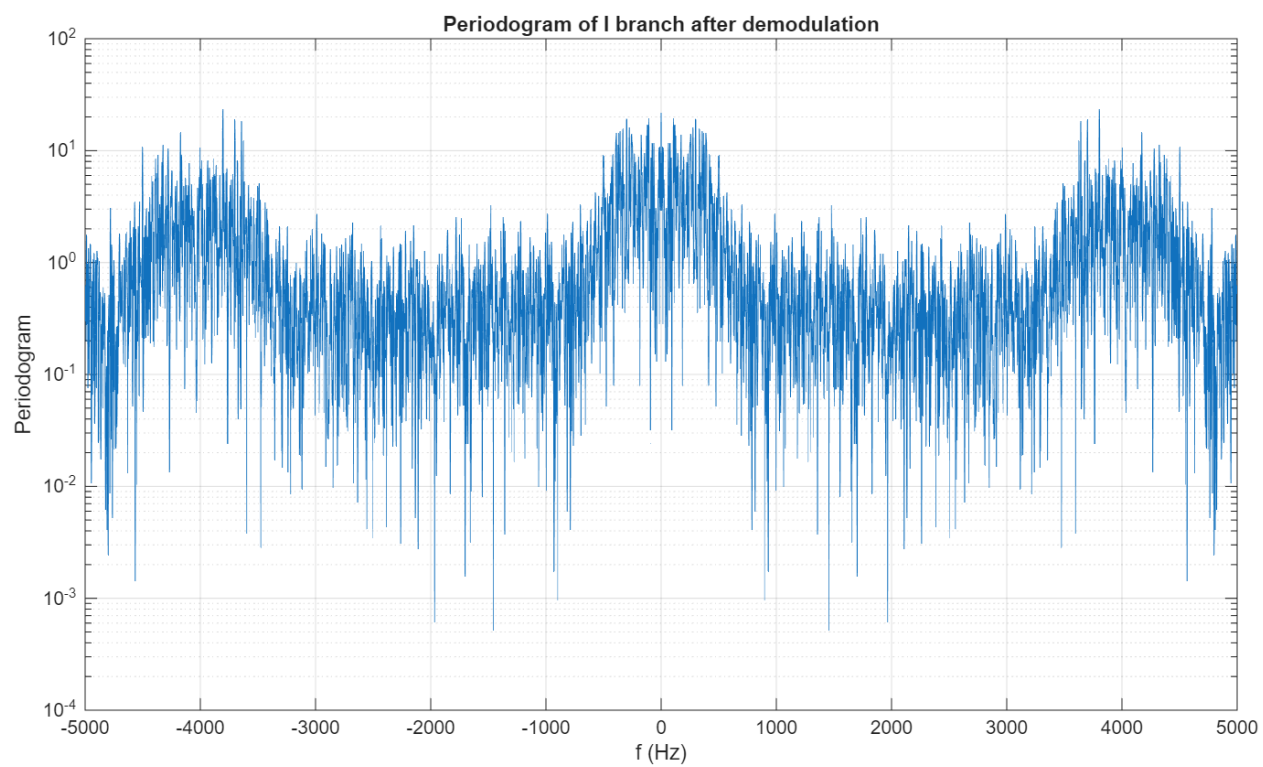
figure;
semilogy(F, PYI_demod);
grid on;
xlabel('f (Hz)'); ylabel('Periodogram');
title('Periodogram of I branch after demodulation');

%% Periodogram of Q branch
YQ_demod_f = Ts * fftshift(fft(YQ_demod, Nf));
Ttotal = length(YQ_demod) * Ts;
PYQ_demod = abs(YQ_demod_f).^2 / Ttotal;

figure;
semilogy(F, PYQ_demod);
grid on;
xlabel('f (Hz)'); ylabel('Periodogram');
title('Periodogram of Q branch after demodulation');
```

After multiplication with the receiver carriers, the passband signal is shifted back toward baseband. However, additional high-frequency components also appear due to the multiplication with the carrier. These high-frequency terms will be removed by the matched filter in the next step. The noise is also present in both branches, so the resulting waveforms are noisier than the original baseband signals.





10. (5) Pass the waveforms computed in the previous step through the matched filters. Plot the resulting waveforms and their periodograms (**set the correct time axis**). What do you observe?

MATLAB code:

```
%% 10. Matched filters
YI_mf = conv(YI_demod, phi) * Ts;
YQ_mf = conv(YQ_demod, phi) * Ts;

t_mf = t(1) + t_phi(1) + (0:length(YI_mf)-1)*Ts;

figure;
plot(t_mf, YI_mf);
grid on;
xlabel('t (sec)'); ylabel('Y_I after MF');
title('I branch after matched filter');

figure;
plot(t_mf, YQ_mf);
grid on;
xlabel('t (sec)'); ylabel('Y_Q after MF');
title('Q branch after matched filter');

%% Periodogram of I branch after matched filter
YI_mf_f = Ts * fftshift(fft(YI_mf, Nf));
Ttotal = length(YI_mf) * Ts;
PYI_mf = abs(YI_mf_f).^2 / Ttotal;

figure;
semilogy(F, PYI_mf);
grid on;
xlabel('f (Hz)'); ylabel('Periodogram');
title('Periodogram of I branch after matched filter');

%% Periodogram of Q branch after matched filter
YQ_mf_f = Ts * fftshift(fft(YQ_mf, Nf));
Ttotal = length(YQ_mf) * Ts;
PYQ_mf = abs(YQ_mf_f).^2 / Ttotal;

figure;
semilogy(F, PYQ_mf);
grid on;
xlabel('f (Hz)'); ylabel('Periodogram');
title('Periodogram of Q branch after matched filter');
```

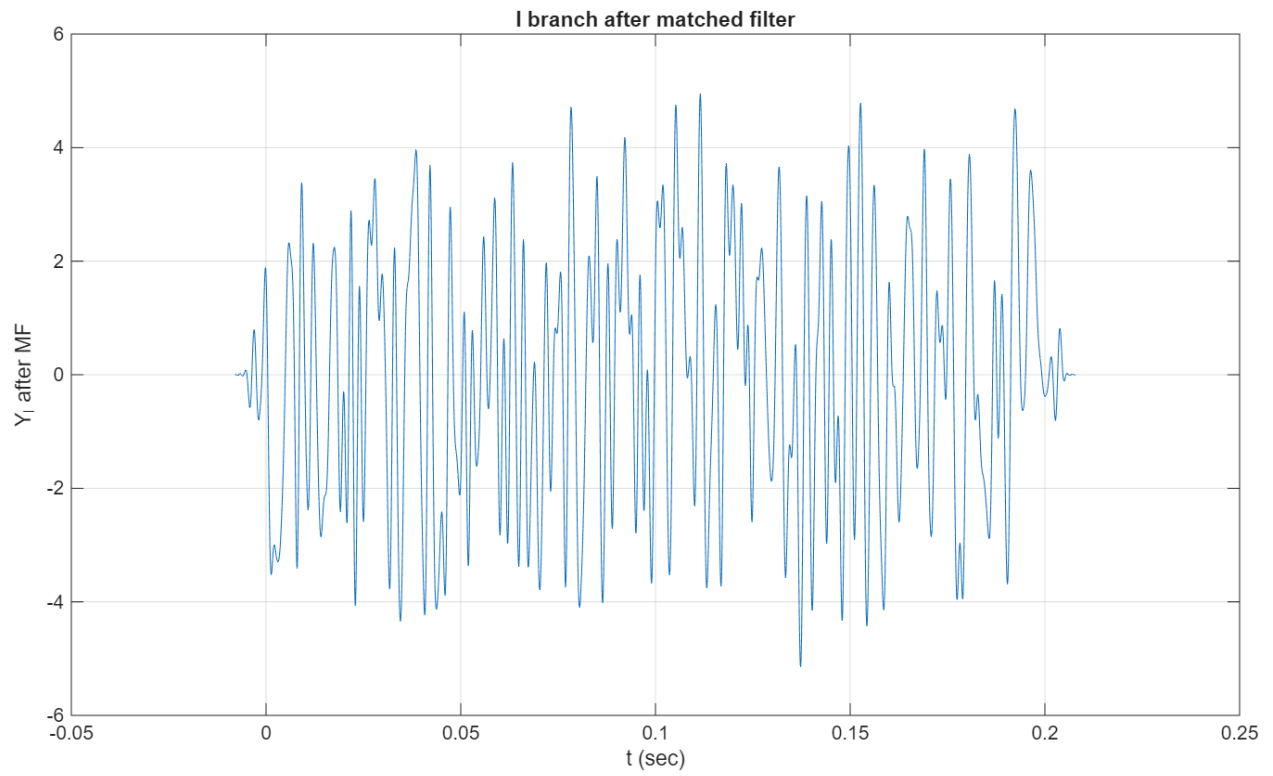
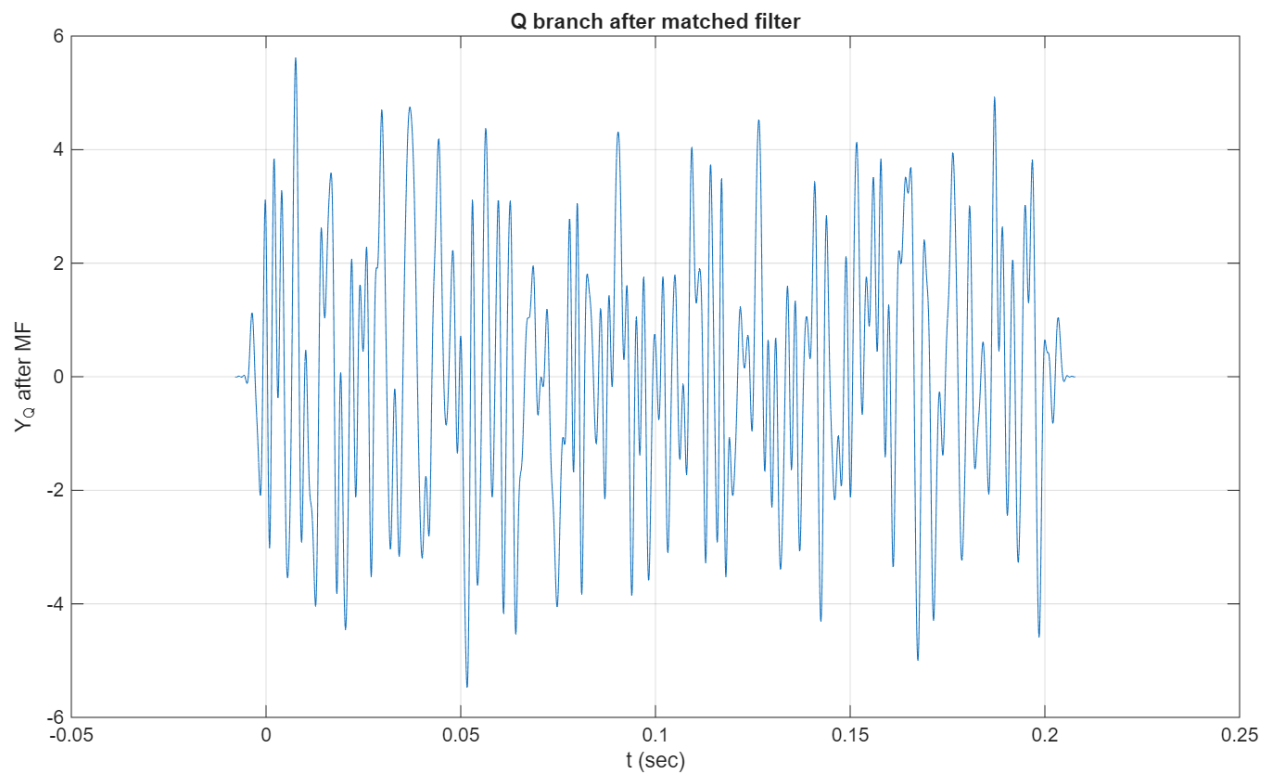
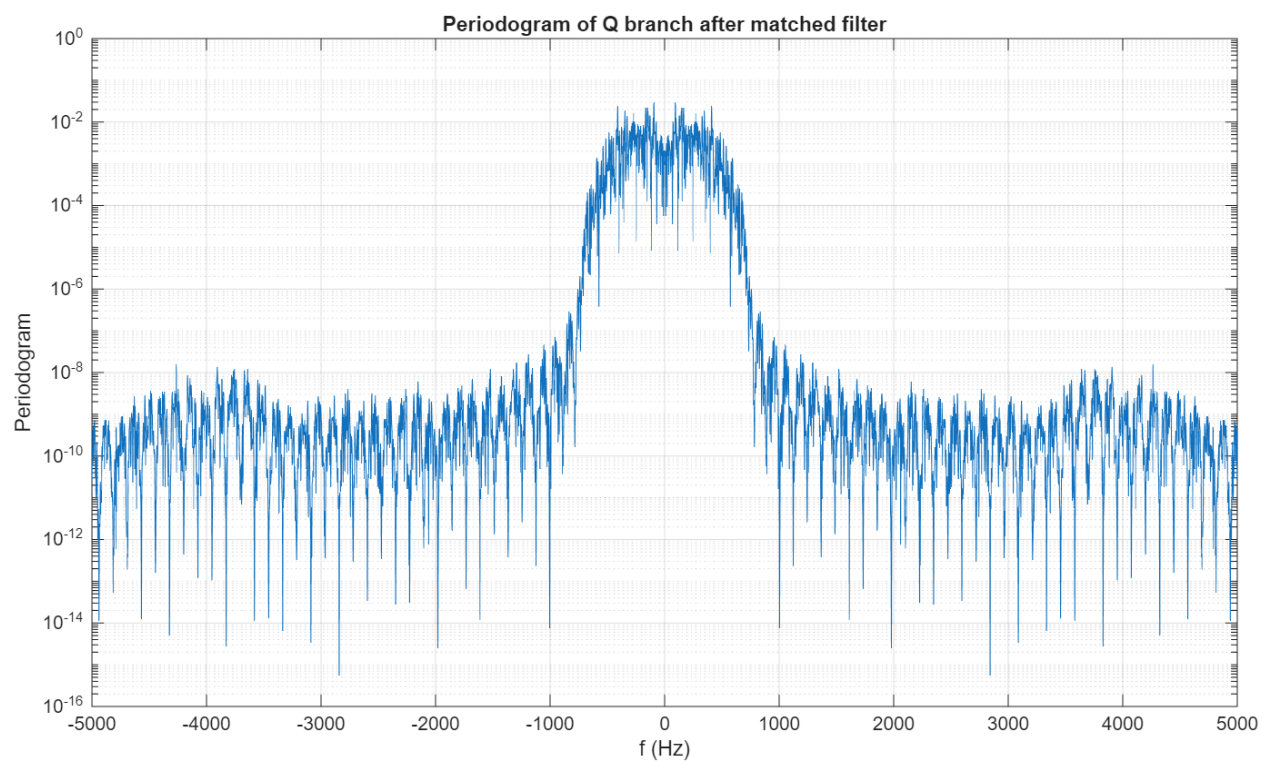
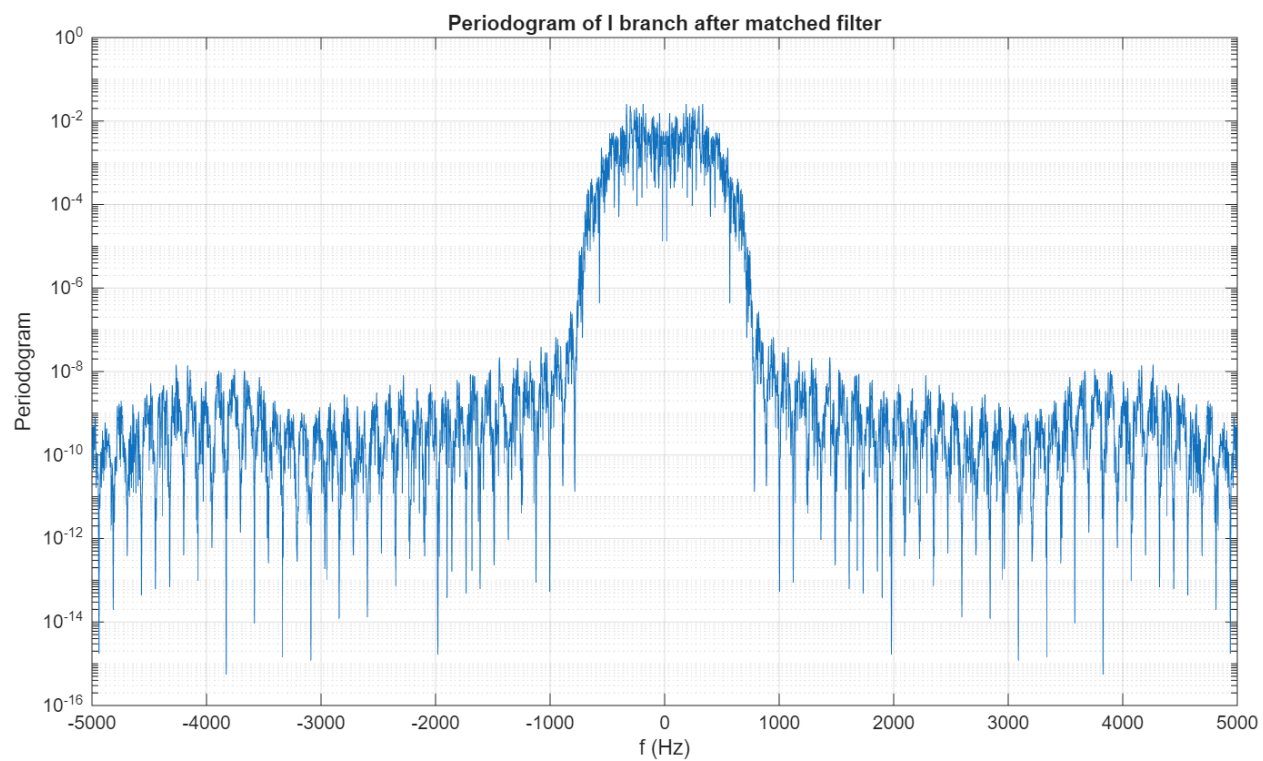



Figure 6. Average periodogram estimate for 4-PAM compared with the theoretical PSD.





After the matched filters, the high-frequency components produced by the demodulation are strongly attenuated, and the useful baseband components are preserved. The resulting signals are smoother and are suitable for sampling at the symbol instants. The periodograms are now mainly concentrated around baseband, while the noise is reduced by the filtering operation.

11. (5) Sample the output of the matched filters at the appropriate time instants and plot the output sequence using the `scatterplot` command.

MATLAB code:

```
% 11. Sampling
delay = 2*A*over;           % total delay of TX SRRC + matched filter

sample_idx = delay + 1 : over : delay + N*over;

YI_samp = YI_mf(sample_idx);
YQ_samp = YQ_mf(sample_idx);

figure;
scatterplot(YI_samp + 1j*YQ_samp);
title('Received 16-QAM constellation after matched filtering');
```

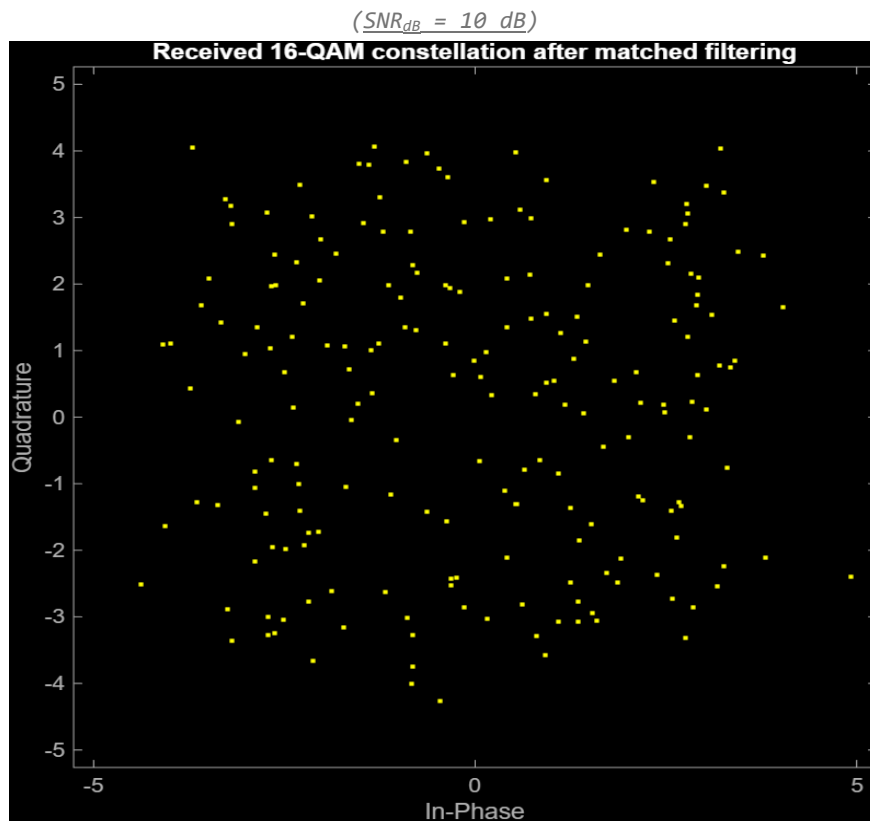
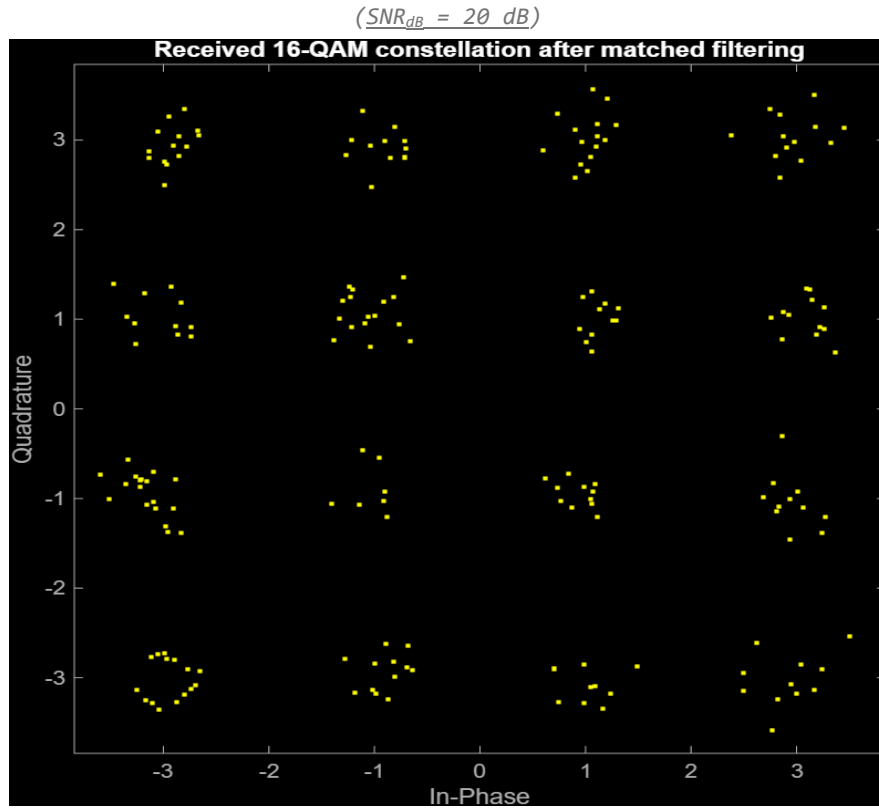


Figure 18. Received 16-QAM constellation after sampling at the matched-filter outputs.

12. (10) Write a function

`function est_X = detect_4_PAM(Y, A)`

which uses the nearest-neighbor rule and makes decisions for the input sequence of 4-PAM symbols on a symbol-by-symbol basis. Apply the function separately to the samples obtained at the inphase output and at the quadrature output, and obtain decisions for the corresponding input sequences.

MATLAB code:

```
function est_X = detect_4PAM(Y, A)
    est_X = zeros(size(Y));

    for k = 1:length(Y)
        if Y(k) < -2*A
            est_X(k) = -3*A;
        elseif Y(k) >= -2*A && Y(k) < 0
            est_X(k) = -A;
        elseif Y(k) >= 0 && Y(k) < 2*A
            est_X(k) = A;
        else
            est_X(k) = 3*A;
        end
    end
end
```

13. (10) Using the input sequences and the decisions, compute the **number of symbol decision errors for the 16-QAM constellation.**

MATLAB code:

```
% 13. Symbol errors for 16-QAM
symbol_errors = sum((XI_est ~= XI_sym) | (XQ_est ~= XQ_sym));
SER = symbol_errors / N;

fprintf('Number of 16-QAM symbol errors = %d out of %d symbols\n', symbol_errors, N);
fprintf('Symbol Error Rate = %.5f\n', SER);
```

Results for $SNR_{dB} = 10$ dB :

Number of 16-QAM symbol errors = 42 out of 200 symbols
Symbol Error Rate = 0.21000

Results for $SNR_{dB} = 20$ dB :

Number of 16-QAM symbol errors = 0 out of 200 symbols
Symbol Error Rate = 0.00000

14. (10) Write a function

```
function est_bit = PAM_4_to_bits(X, A)
```

which uses the inverse Gray mapping, that is, the mapping from symbols to pairs of bits, and, from the decisions for the input symbol sequences, computes the estimated input binary sequence.

MATLAB code:

```
function est_bit = PAM_4_to_bits(X, A)
    est_bit = zeros(1, 2*length(X));

    for k = 1:length(X)
        if X(k) == -3*A
            bit_pair = [0 0];
        elseif X(k) == -A
            bit_pair = [0 1];
        elseif X(k) == A
            bit_pair = [1 1];
        elseif X(k) == 3*A
            bit_pair = [1 0];
        end
        est_bit(2*k-1 : 2*k) = bit_pair;
    end
end
```

15. (10) Compute the number of bit errors.

MATLAB code:

```
% 14. Convert detected symbols back to bits
bitsI_est = PAM_4_to_bits(XI_est, Aamp);
bitsQ_est = PAM_4_to_bits(XQ_est, Aamp);

bits_est = [bitsI_est bitsQ_est];

% 15. Bit errors
bit_errors = sum(bits_est ~= bits);
BER = bit_errors / length(bits);

fprintf('Number of bit errors = %d out of %d bits\n', bit_errors, length(bits));
fprintf('Bit Error Rate = %.5f\n', BER);
```

Results for $SNR_{dB} = 10$ dB :

Number of bit errors = 52 out of 800 bits
Bit Error Rate = 0.06500

Results for $SNR_{dB} = 20$ dB :

Number of bit errors = 0 out of 800 bits
Bit Error Rate = 0.00000

MONTE CARLO METHOD

In the second part, you will estimate the symbol and bit error probabilities using the Monte Carlo method.

1. For $\text{SNR}_{\text{dB}} = [0 : 2 : 16]$, experimentally compute the symbol-decision and bit error probabilities by repeating the steps above K times (for example, $K = 200, 1000$) for each SNR. The experimental estimate of the symbol error probability for each SNR is computed as follows:

$$\hat{P}(E_{\text{symbol}}) = \frac{\text{total number of symbol decision errors}}{\text{total number of transmitted symbols}},$$

whereas the bit error probability is computed as follows:

$$\hat{P}(E_{\text{bit}}) = \frac{\text{total number of bit decision errors}}{\text{total number of transmitted bits}}.$$

MATLAB code:

```
clear; close all; clc;

%% Parameters
N = 200;
K = 200; % K times
Aamp = 1;

T = 10^(-3);
over = 10;
Ts = T/over;
F0 = 2000;

A = 4;
a = 0.5;

SNRdB_vec = 0:2:16;

[phi, t_phi] = srrc_pulse(T, over, A, a);

SER_exp = zeros(size(SNRdB_vec));
BER_exp = zeros(size(SNRdB_vec));

for i = 1:length(SNRdB_vec)

    SNRdB = SNRdB_vec(i);

    total_symbol_errors = 0;
    total_bit_errors = 0;

    total_symbols = K*N;
    total_bits = K*4*N;

    for kk = 1:K
        bits = randi([0 1], 1, 4*N);

        bitsI = bits(1:2*N);
```

```

bitsQ = bits(2*N+1:end);

XI_sym = bits_to_4PAM(bitsI, Aamp);
XQ_sym = bits_to_4PAM(bitsQ, Aamp);

% Pulse shaping
XI_delta = 1/Ts * upsample(XI_sym, over);
XQ_delta = 1/Ts * upsample(XQ_sym, over);

XI_t = conv(XI_delta, phi) * Ts;
XQ_t = conv(XQ_delta, phi) * Ts;

t = t_phi(1) + (0:length(XI_t)-1)*Ts;

% Modulation
XI_mod = 2 * XI_t .* cos(2*pi*F0*t);
XQ_mod = -2 * XQ_t .* sin(2*pi*F0*t);

Xmod = XI_mod + XQ_mod;

% Noise
sigmaW2 = 10*Aamp^2 / (Ts * 10^(SNRdB/10));
W = sqrt(sigmaW2) * randn(size(Xmod));

Y = Xmod + W;

% Demodulation
YI_demod = Y .* cos(2*pi*F0*t);
YQ_demod = Y .* (-sin(2*pi*F0*t));

% Matched filter
YI_mf = conv(YI_demod, phi) * Ts;
YQ_mf = conv(YQ_demod, phi) * Ts;

t_mf = t(1) + t_phi(1) + (0:length(YI_mf)-1)*Ts;

% Sampling
sample_times = (0:N-1)*T;
sample_idx = zeros(1,N);

for n = 1:N
    [~, sample_idx(n)] = min(abs(t_mf - sample_times(n)));
end

YI_samp = YI_mf(sample_idx);
YQ_samp = YQ_mf(sample_idx);

% Detection
XI_est = detect_4PAM(YI_samp, Aamp);
XQ_est = detect_4PAM(YQ_samp, Aamp);

% Symbol errors
symbol_errors = sum((XI_est ~= XI_sym) | (XQ_est ~= XQ_sym));
total_symbol_errors = total_symbol_errors + symbol_errors;

```



```

    % Bit errors
    bitsI_est = PAM_4_to_bits(XI_est, Aamp);
    bitsQ_est = PAM_4_to_bits(XQ_est, Aamp);

    bits_est = [bitsI_est bitsQ_est];

    bit_errors = sum(bits_est ~= bits);
    total_bit_errors = total_bit_errors + bit_errors;
end

SER_exp(i) = total_symbol_errors / total_symbols;
BER_exp(i) = total_bit_errors / total_bits;

end

%% Theoretical probabilities
SNRlin = 10.^(SNRdB_vec/10);
q_arg = sqrt(SNRlin/5);
Pdim = (3/2) * Q(q_arg);

SER_theory = 1 - (1 - Pdim).^2;
BER_theory_approx = (3/4) * Q(q_arg);

%% SER plot
figure;
semilogy(SNRdB_vec, SER_exp, 'o-', 'LineWidth', 1.5); hold on;
semilogy(SNRdB_vec, SER_theory, 's-', 'LineWidth', 1.5);
grid on;
xlabel('SNR_{dB}');
ylabel('Symbol Error Probability');
legend('Experimental SER', 'Theoretical SER');
title('Symbol error probability for 16-QAM');

%% BER plot
figure;
semilogy(SNRdB_vec, BER_exp, 'o-', 'LineWidth', 1.5); hold on;
semilogy(SNRdB_vec, BER_theory_approx, 's-', 'LineWidth', 1.5);
grid on;
xlabel('SNR_{dB}');
ylabel('Bit Error Probability');
legend('Experimental BER', 'Theoretical BER approximation');
title('Bit error probability for 16-QAM');

```

2. (20) Plot, using semilogy, the theoretical and experimental symbol error probabilities as a function of SNR_{dB} . What do you observe?

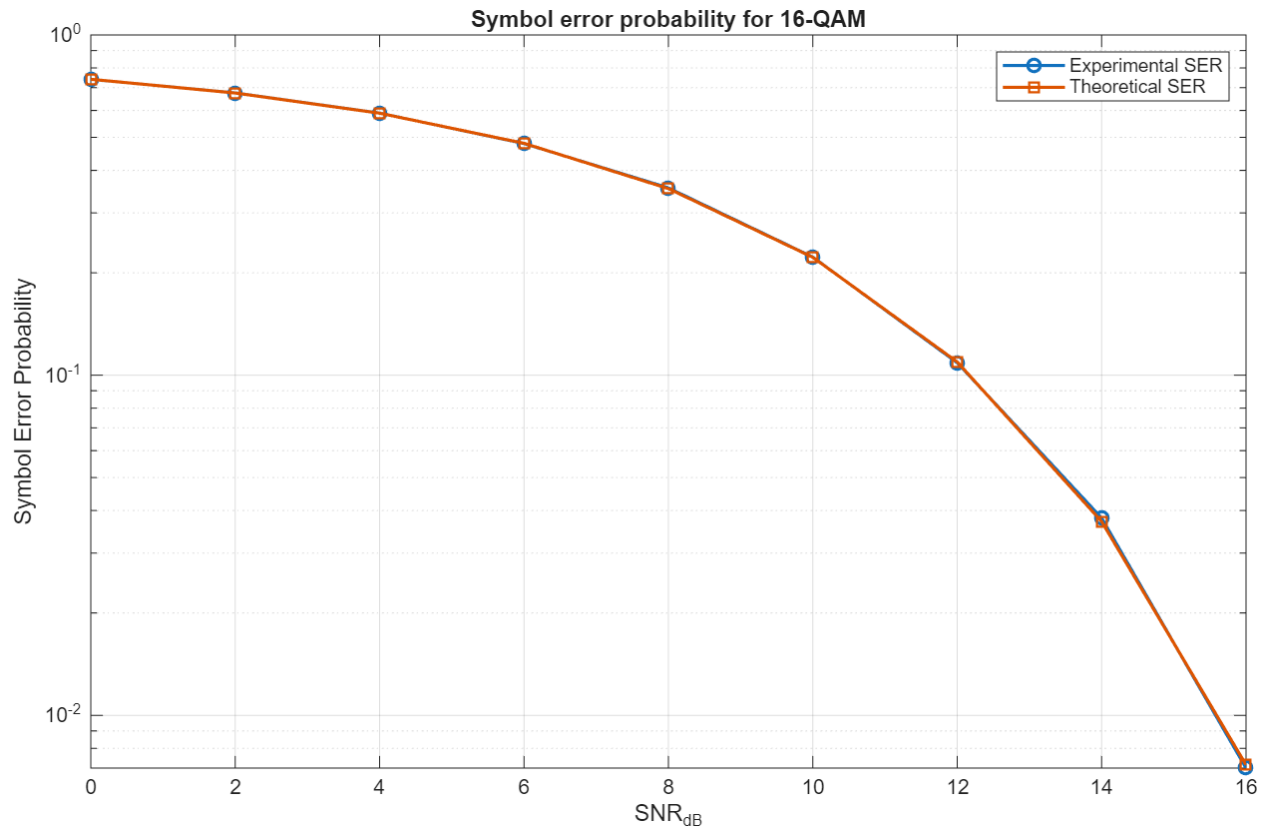


Figure 1. Experimental and theoretical symbol error probability versus SNR.

The experimental symbol error probability decreases as the SNR increases. The experimental curve follows the theoretical curve quite closely, especially for low and medium SNR values. Small differences are expected because the Monte Carlo simulation uses a finite number of transmitted symbols. At high SNR, the number of errors becomes very small, so a larger number of repetitions K is required for a more accurate estimate.

3. (20) Plot, using semilogy, the experimental bit error probability as a function of SNR_{dB} . In the same figure, also plot the theoretical approximation for the bit error probability. What do you observe? Can you explain the phenomenon?

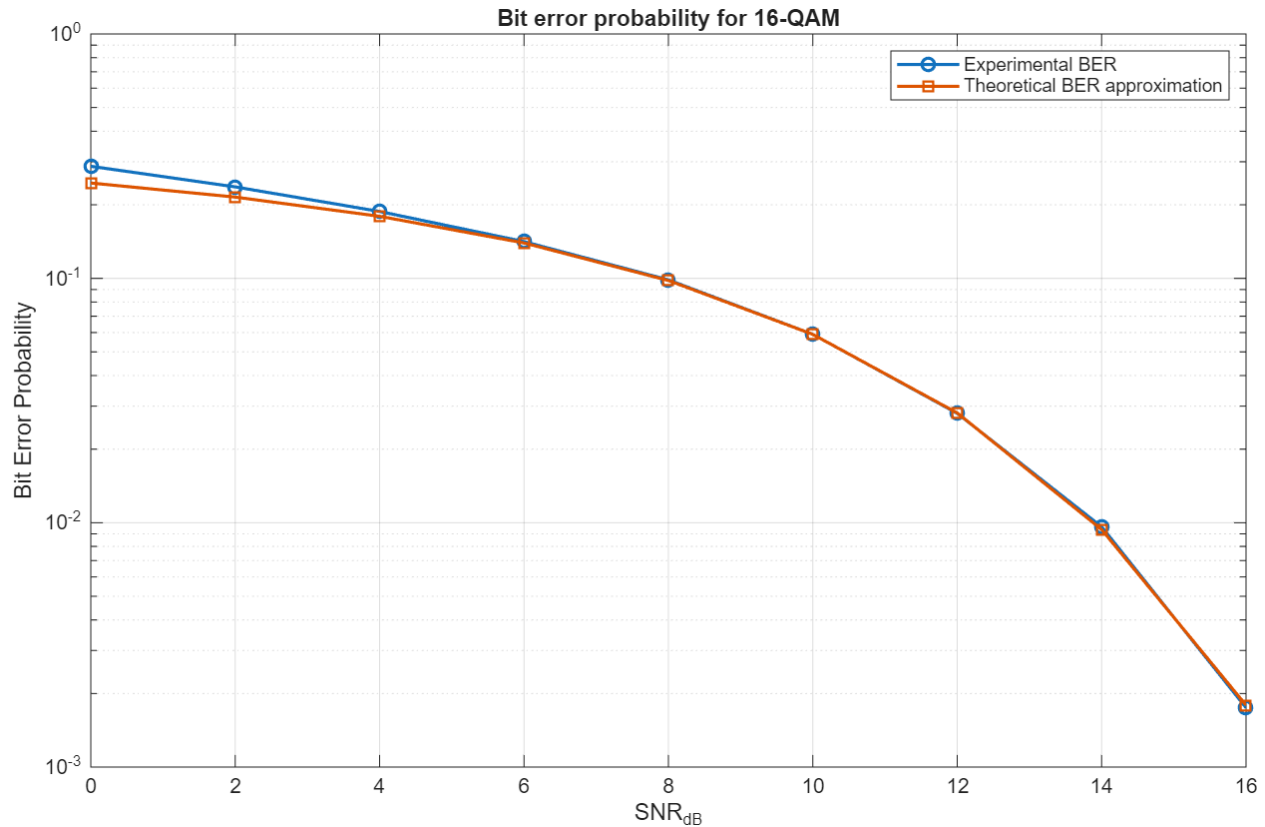


Figure 2. Experimental and theoretical symbol error probability versus SNR.

The bit error probability also decreases as the SNR increases. The experimental BER follows the theoretical approximation, but small deviations appear due to the finite number of Monte Carlo repetitions. The approximation becomes more accurate at higher SNR values, where most symbol errors are caused by decisions to nearest-neighbor constellation points. Because Gray coding is used, most symbol errors cause only one bit error, which explains why the BER is smaller than the SER.